The Internet of Things: Connecting Our World

Session-3: (IoT Stack practical vs theory) Detailed Explained

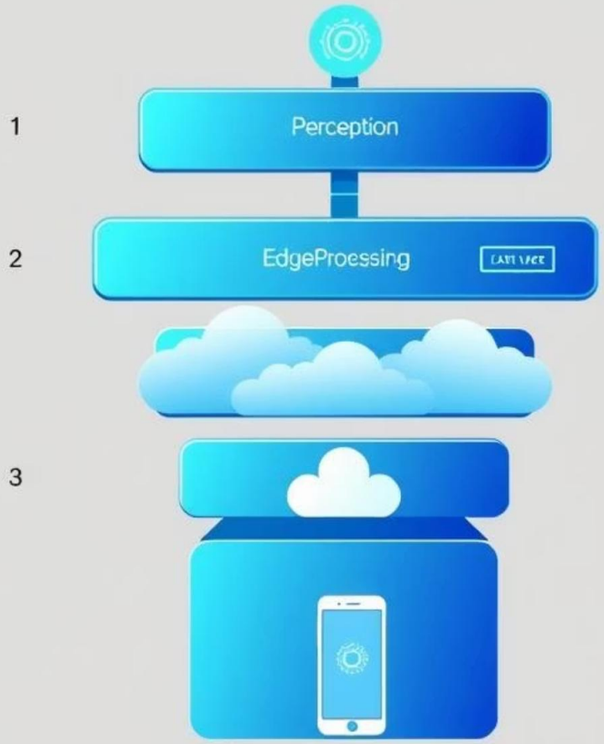
Eng.Omar Sameh

Understanding IoT Stack in Detail

The IoT (Internet of Things) stack represents the various layers of technology that work together to enable connected devices and systems. Think of it like a multi-layered cake or a set of stairs, where each layer plays a crucial role in the overall functionality.

In the following sections, we will explore:

- The IoT stack from both theoretical and practical perspectives.
- How each layer connects and interacts to form complete IoT solutions.
- Real-world examples and implementations that demonstrate these concepts.



IoT Stack (Theoretical Model)

The IoT Stack, in its theoretical model, breaks down the complex architecture of an Internet of Things system into five distinct, interacting layers. This layered approach helps to understand the different functionalities and technologies involved in connecting physical devices to the digital world and delivering meaningful services to users.

Application Layer

What users see - mobile app controlling lights, dashboard showing readings, alerts on phone

Middleware/Platform Layer

Connects devices to cloud, includes Message Brokers like MQTT, platforms like Firebase, AWS IoT, Azure IoT - acts as "translator" between devices and applications

Edge/Processing Layer

Intermediate devices (Gateway or Edge Device) for initial processing, like Raspberry Pi or ESP32 for filtering or compressing data

Network Layer (Communication Layer)

All communication protocols: Wi-Fi, Bluetooth, Zigbee, LoRa, NB-IoT - transfers data from devices to outside world

Perception Layer (Hardware Layer)

Sensors, actuators, cameras - devices that "see/sense" the real world

This structured model helps in designing, implementing, and troubleshooting IoT solutions by clearly defining the responsibilities and interfaces between different components.

Eng.Omar Sameh

1. Perception Layer: The Senses 👀

This is the "senses" of the system.

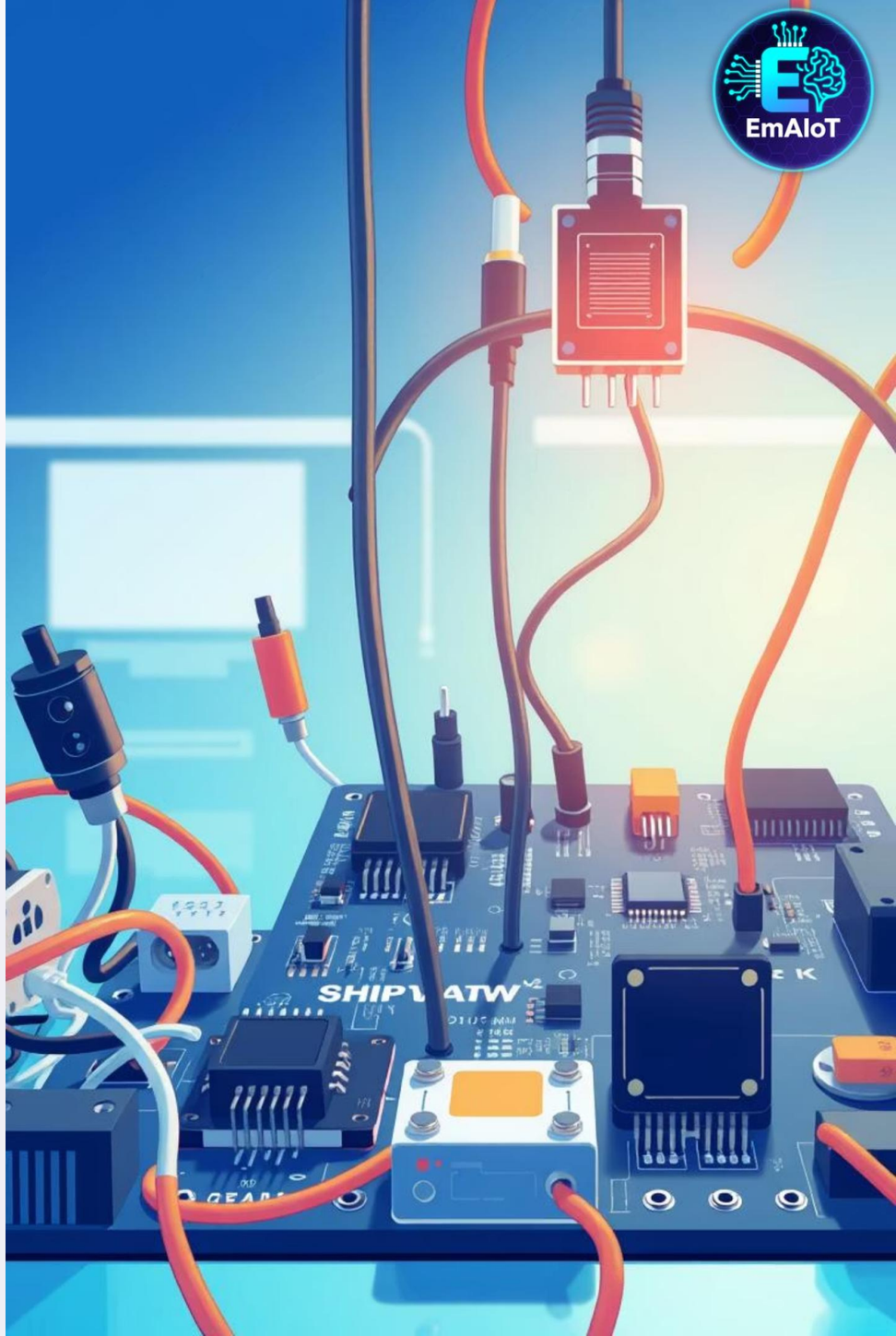
It takes information from the environment (temperature, humidity, movement, light).

This layer consists of:

- **Sensors:** Get the data (DHT22, BME280, motion sensors)
- **Actuators:** Execute commands (lights, motors, relays)

Real examples include a temperature sensor reading 25°C, or a motion detector sensing movement.

Ultimately, this layer is the physical interface with the real world.



2. Network Layer: The Highway 🛣️

This is the "road" that data travels on.

Responsible for sending data from sensor → gateway → cloud.

This layer includes communication technologies:

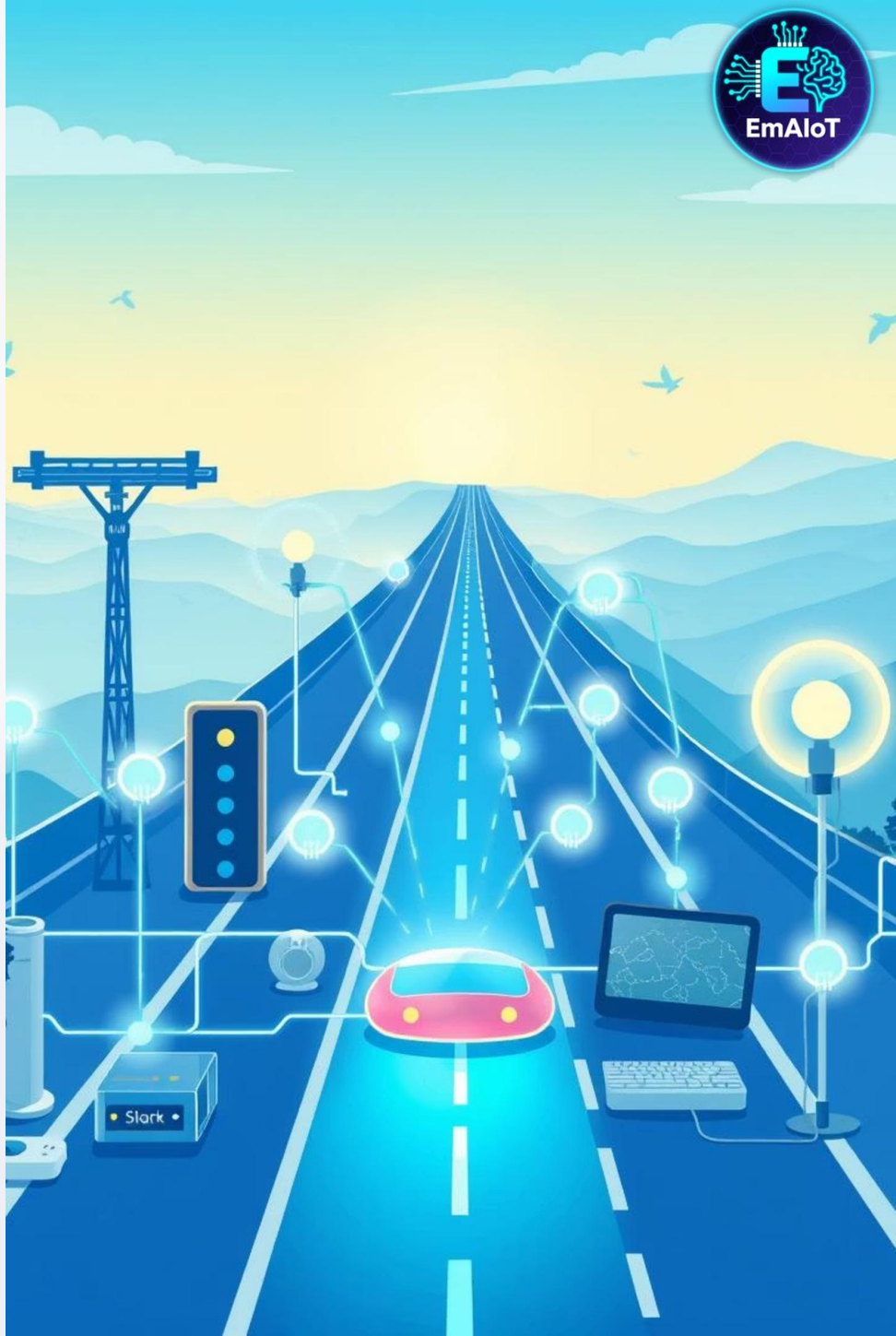
- Wi-Fi, Bluetooth, Zigbee, LoRa, NB-IoT, 4G/5G

It also includes transmission protocols:

- MQTT, CoAP, HTTP

Real example: ESP32 sends temperature data via MQTT over Wi-Fi to cloud server.

Chooses best path based on distance, power, and data requirements.



3. Edge Layer: The Smart Secretary ✍️

This is an intermediate station.

Like a "secretary" that collects and filters data before it goes to cloud.

Makes quick local decisions: "If temperature > 30°C → turn on fan immediately".

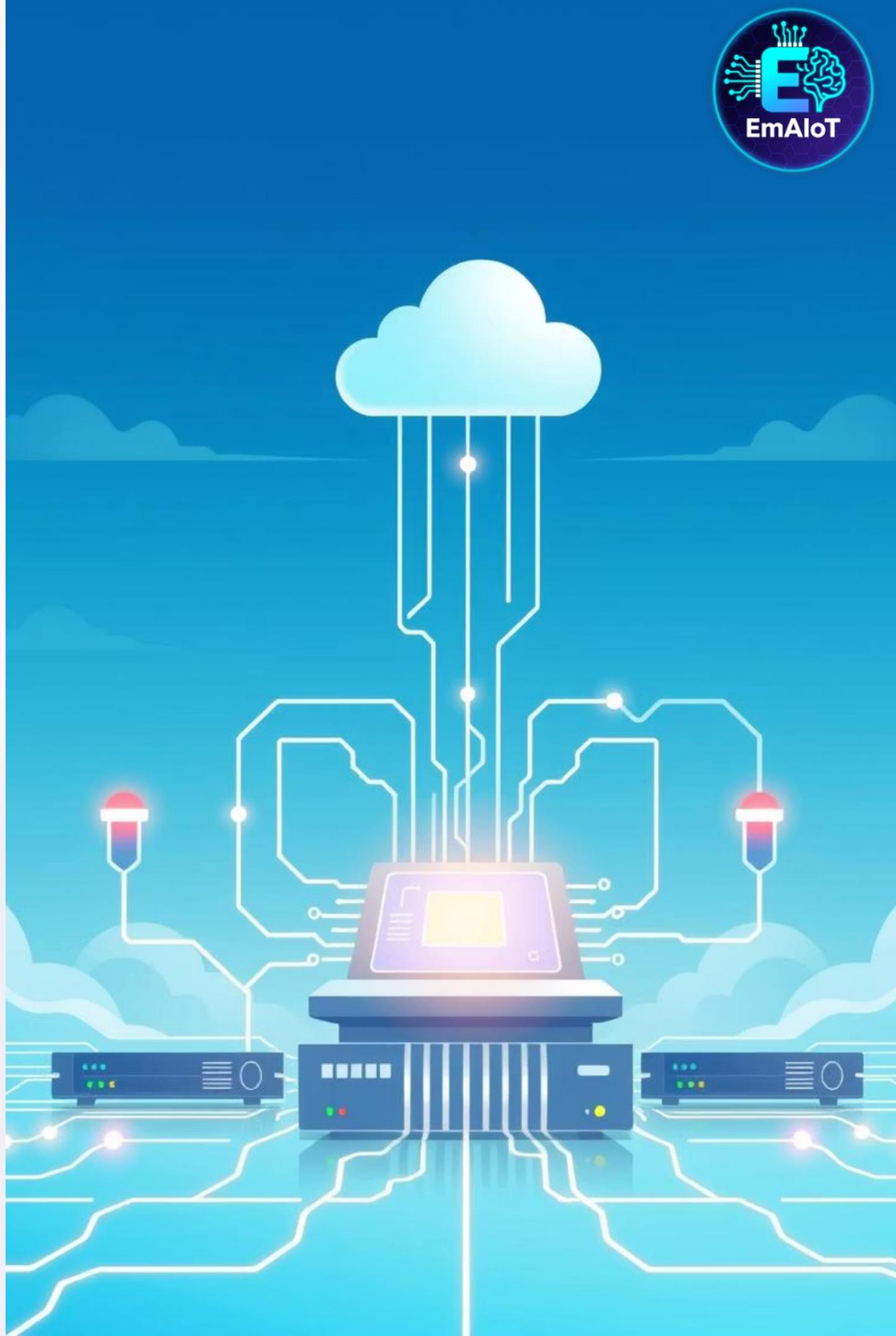
Examples of edge devices:

- Raspberry Pi as gateway
- ESP32 with local processing
- Smart home hubs

Benefits:

- Faster response
- Reduced cloud traffic
- Works offline

Real example: Smart thermostat adjusts temperature locally without waiting for cloud.



4. Middleware/Platform Layer: The Brain 🧠

This is the "brain" in the cloud.

Main functions:

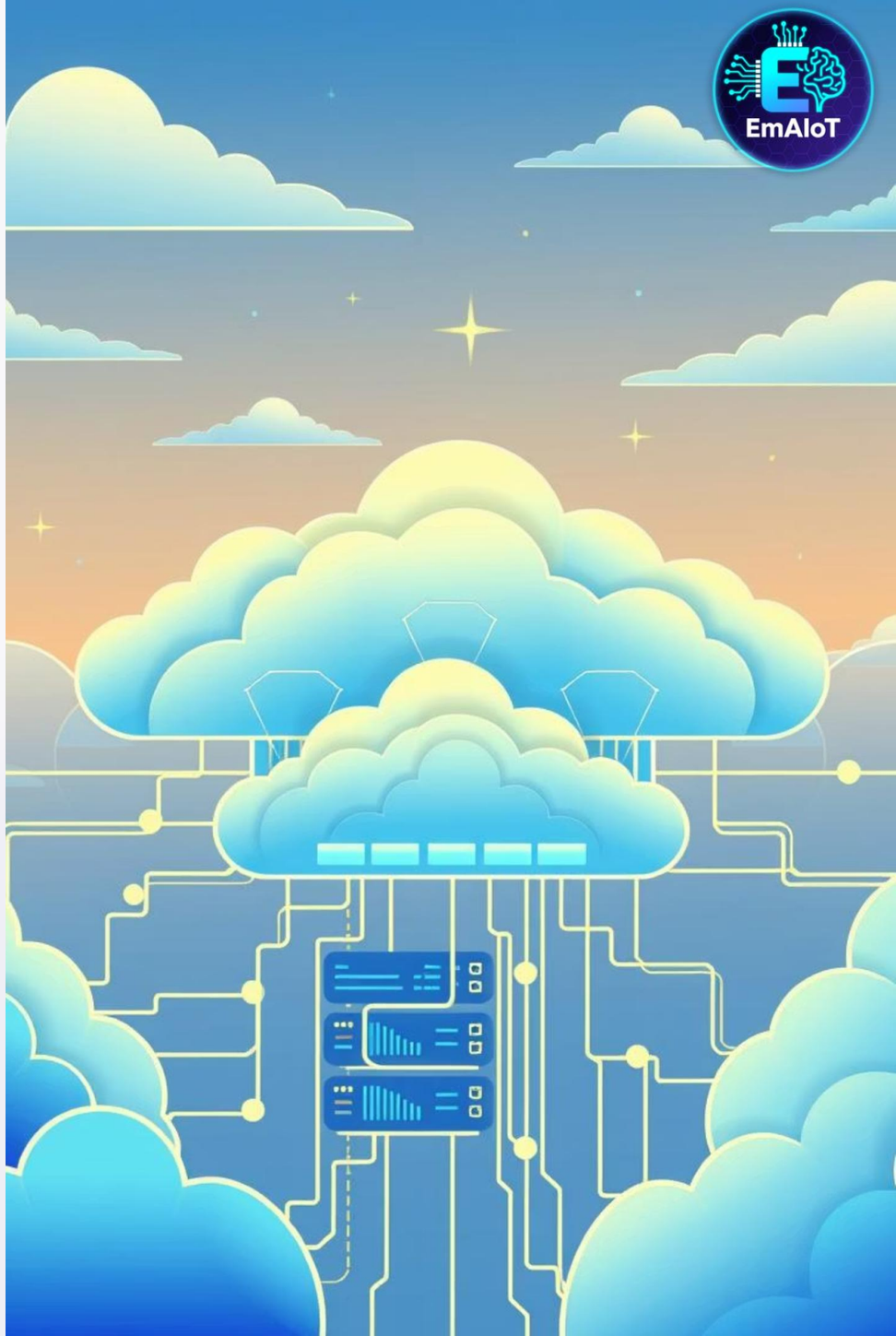
- **Data storage:** in databases
- **Device management:** and monitoring
- **Rules Engine:** execution
- **Analytics:** and Machine Learning

Examples of platforms:

- AWS IoT Core, Azure IoT Hub
- Firebase, ThingsBoard
- Self-hosted solutions

Real example: Platform receives temperature data, stores it, and triggers SMS alert if > 35°C.

Handles millions of devices and messages.



5. Application Layer: What You See

This is what you actually see and interact with

Types of applications:

- Mobile Apps (iOS/Android)
- Web Dashboards (Grafana, custom)
- Notifications (SMS, Email, Push)
- Voice assistants (Alexa, Google)

Main purposes:

- Display information in user-friendly way
- Allow user control of devices
- Send alerts and notifications

Real examples: Smart home app, temperature dashboard, security alerts

The final interface between IoT system and humans

IoT Stack (Practical Implementation)

Cloud Applications

Customer interface for dashboards, settings and device control

Cloud Platform

Where data from IoT devices is captured, processed and stored (e.g. Amazon Web Services, Microsoft Azure, Google Cloud)

Communications

How devices connect to the internet and transfer data (Wired, Bluetooth, WiFi, Zigbee, Thread, LTE-M/NB-IoT/2G/3G/4G/5G cellular networks, LoRaWAN)

Device Software

Runs on the device's processor and controls its functionality (Languages used include: Linux, C, Rust, Python, C#)

Device Hardware

Embedded into "Things" in the IoT (ESP32, ARM Cortex, SoCs, gateways, PLCs, interface modules and connectors)

Things

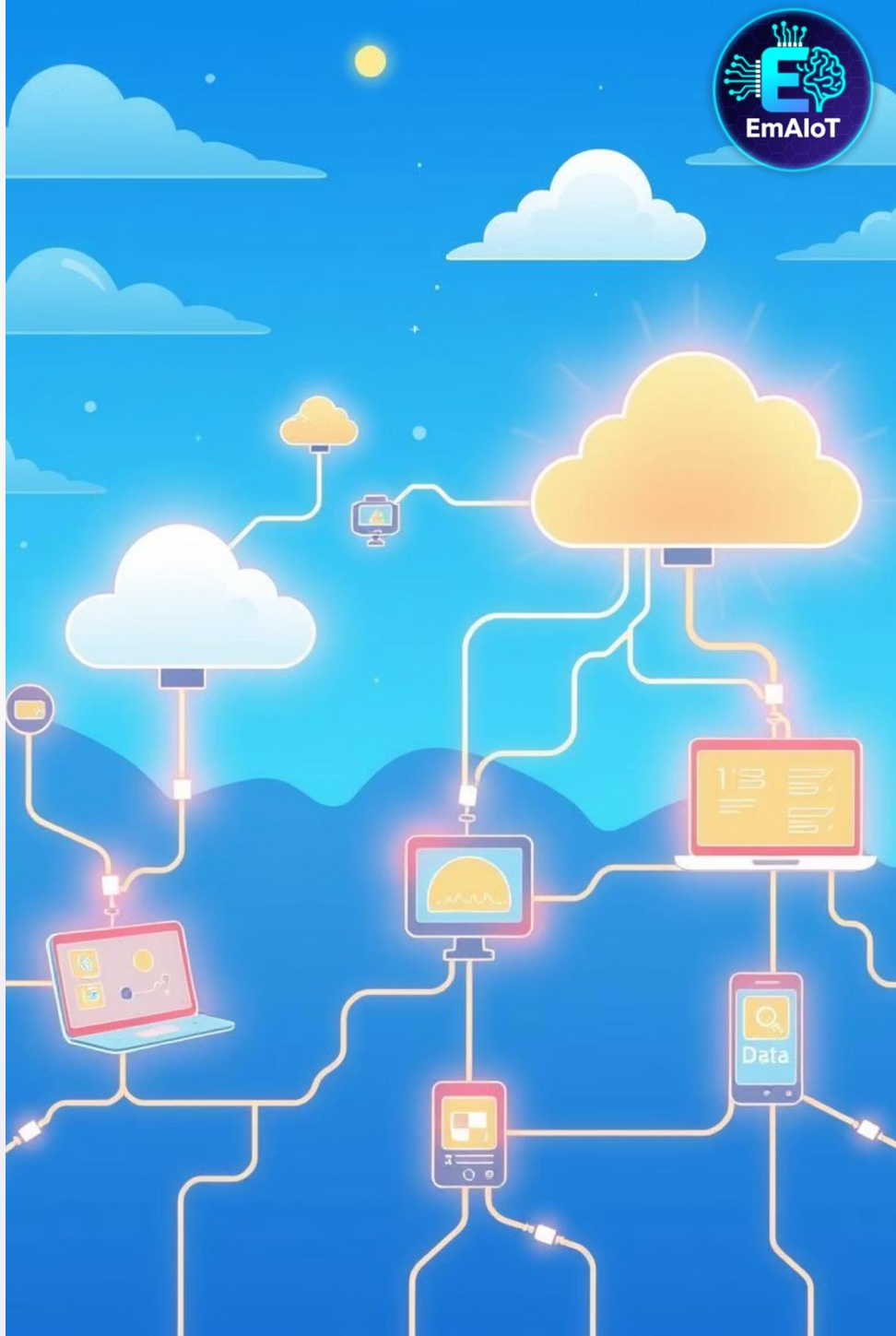
Physical objects found around home, work and everyday life

IoT Stack (Practical Implementation) - Detailed Breakdown

Now we'll explore each layer of the practical IoT Stack in detail, tracing the path from physical "Things" to Cloud Applications.

This section will cover:

- Real-world examples and implementations
- How each layer works in practice with actual devices and technologies
- The journey of data from sensor to user interface



1. Things: The Physical World 🏠

Any object you want to make smart: lights, fans, sensors, machines.

Real examples:

- Temperature sensor (DHT22)
- Smart light bulb
- Security camera
- Industrial machine

These are "dumb" objects that become "smart" when connected.

Examples in practice:

- Home: Smart thermostat, door locks, lights
- Office: Meeting room sensors, air quality monitors
- Industry: Vibration sensors on machines, temperature monitors

The starting point of every IoT system.

2. Device Hardware: The Physical Components

The physical components that make devices "smart" are the foundation of any IoT system. This layer includes microcontrollers, sensors, actuators, power sources, and connectivity modules, all working together to enable a device to interact with the real world and transmit data.

MCU (Microcontroller)

ESP32, STM32, Arduino: The brains of the operation, executing code and managing peripherals to control the device's functions.

Sensors

DHT22, BME280, PIR motion sensors: These components detect and collect data from the environment, converting physical parameters into electrical signals.

Actuators

Relays, motors, LEDs: Components that translate electrical signals into physical actions or visual feedback, allowing the device to influence its environment.

Power

Battery, USB, AC adapter: Essential for supplying the necessary energy for the device to operate, depending on portability and deployment needs.

Connectivity

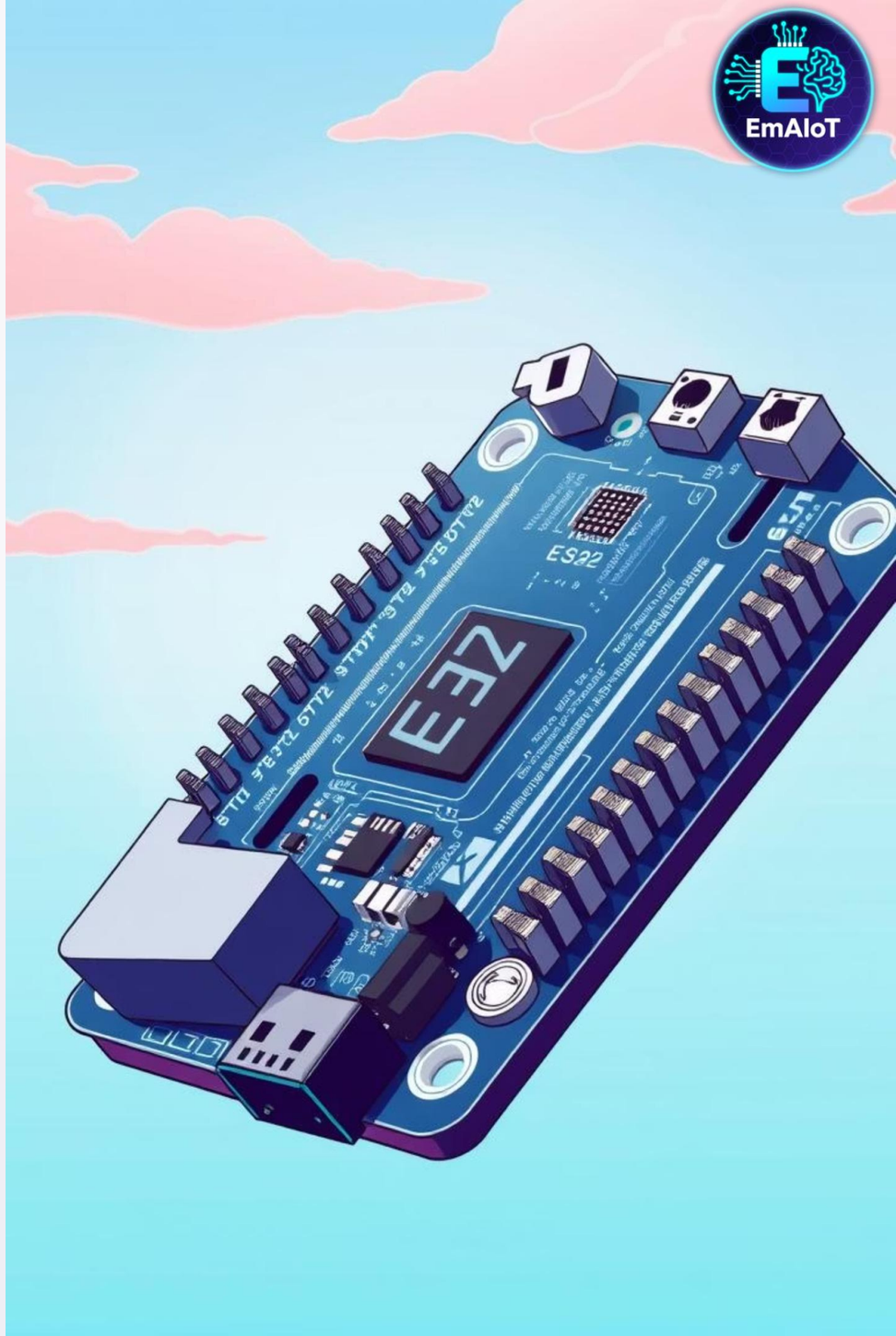
WiFi chip, Bluetooth module, LoRa radio: These modules enable communication between the device and local networks or the internet, facilitating data exchange.

Real Example: ESP32 Board

The ESP32 is a popular, versatile microcontroller that often integrates many of these components, making it ideal for rapid IoT development:

- Built-in WiFi and Bluetooth for seamless wireless communication.
- GPIO pins for connecting a variety of sensors and peripherals.
- ADC for analog readings, converting real-world analog signals into digital data.
- PWM for motor control, enabling precise speed and direction adjustments for actuators.

Decisions made at this layer are crucial, such as choosing between WiFi or LoRa for connectivity, or battery versus mains power, profoundly impacting the device's functionality and deployment.



3. Device Software: The Firmware Brain 💻

This layer refers to the software or firmware that directly runs on the microcontrollers and processors embedded within IoT devices. It's the "brain" that tells the hardware what to do, enabling it to collect data, perform actions, and communicate with other systems.

Common programming languages include: C/C++, Python, Rust, and MicroPython.

Main functions of device software:

- Read sensors (temperature, humidity, motion) to gather data from the physical environment.
- Control actuators (turn on/off lights, motors) to interact with the physical world.
- Connect to network (WiFi, Bluetooth) to establish communication channels.
- Send data to cloud (MQTT, HTTP) for storage, analysis, and further processing.
- Handle security (TLS, certificates) to ensure secure data transmission and device authentication.
- Manage power (sleep modes, wake up) to optimize battery life for low-power devices.

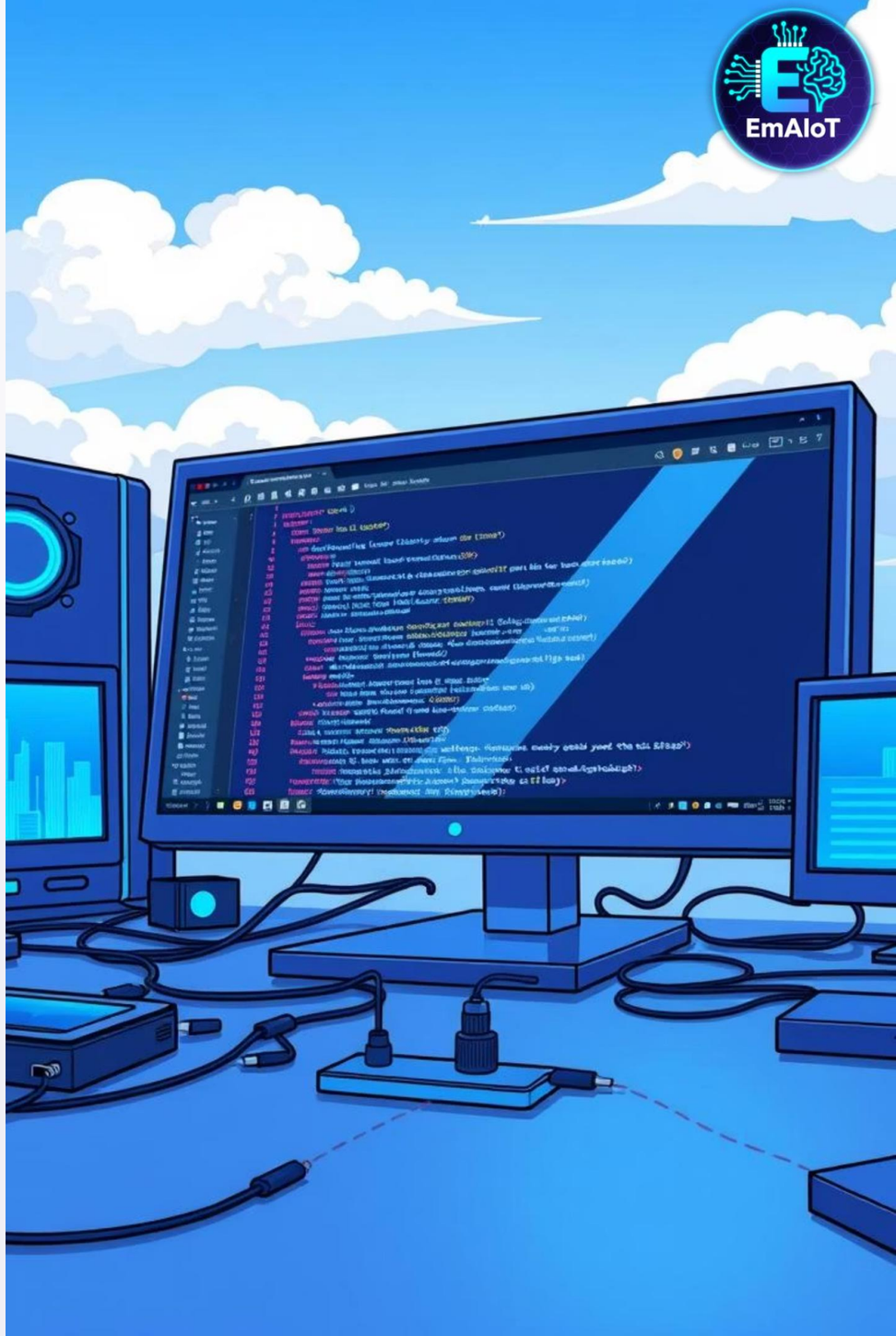
Real Example: ESP32 Firmware

An ESP32-based device's firmware might be programmed to:

- Reads DHT22 sensor every 30 seconds to monitor environmental conditions.
- Sends data via MQTT to "home/sensor/temp" topic, a common protocol for lightweight messaging.
- Goes to deep sleep to save battery, waking up only to perform scheduled tasks.
- Handles OTA (Over-The-Air) updates securely, allowing for remote firmware upgrades without physical access.

Development tools commonly used:

- Arduino IDE (Integrated Development Environment)
- ESP-IDF (Espressif IoT Development Framework)
- PlatformIO



4. Communications: The Data Highway 📡

This layer describes how IoT devices connect to the internet and transfer data, forming the crucial link between the physical world and digital platforms.

Connection types:

- Wired: Ethernet, USB
- Short-range wireless: WiFi, Bluetooth, Zigbee
- Long-range wireless: LoRaWAN, NB-IoT, 4G/5G

Protocol selection is based on:

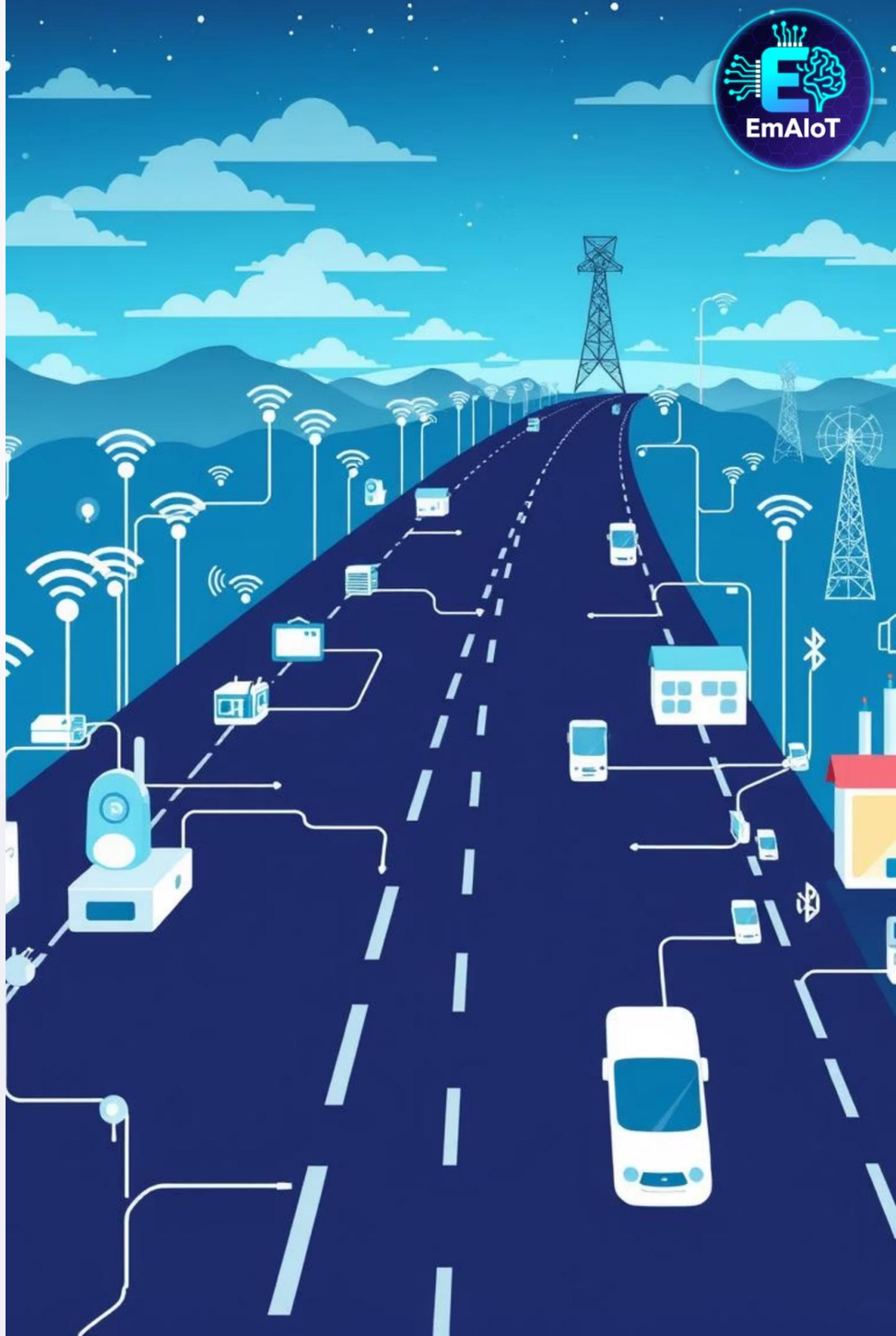
- Distance: WiFi (home), LoRa (city-wide), 4G (global)
- Power consumption: Bluetooth LE (low), WiFi (medium), 4G (high)
- Data rate: Zigbee (low), WiFi (high), 5G (very high)

Real examples:

- Smart home: WiFi + MQTT
- Agricultural sensors: LoRaWAN + CoAP
- Vehicle tracking: 4G + HTTP

Message protocols:

MQTT, HTTP, CoAP, WebSocket



5. Cloud Platform: The Data Processing Center

This is where all the IoT data captured from devices is sent, processed, and stored. It acts as the central hub for managing devices, executing logic, and preparing data for applications.

Main functions:

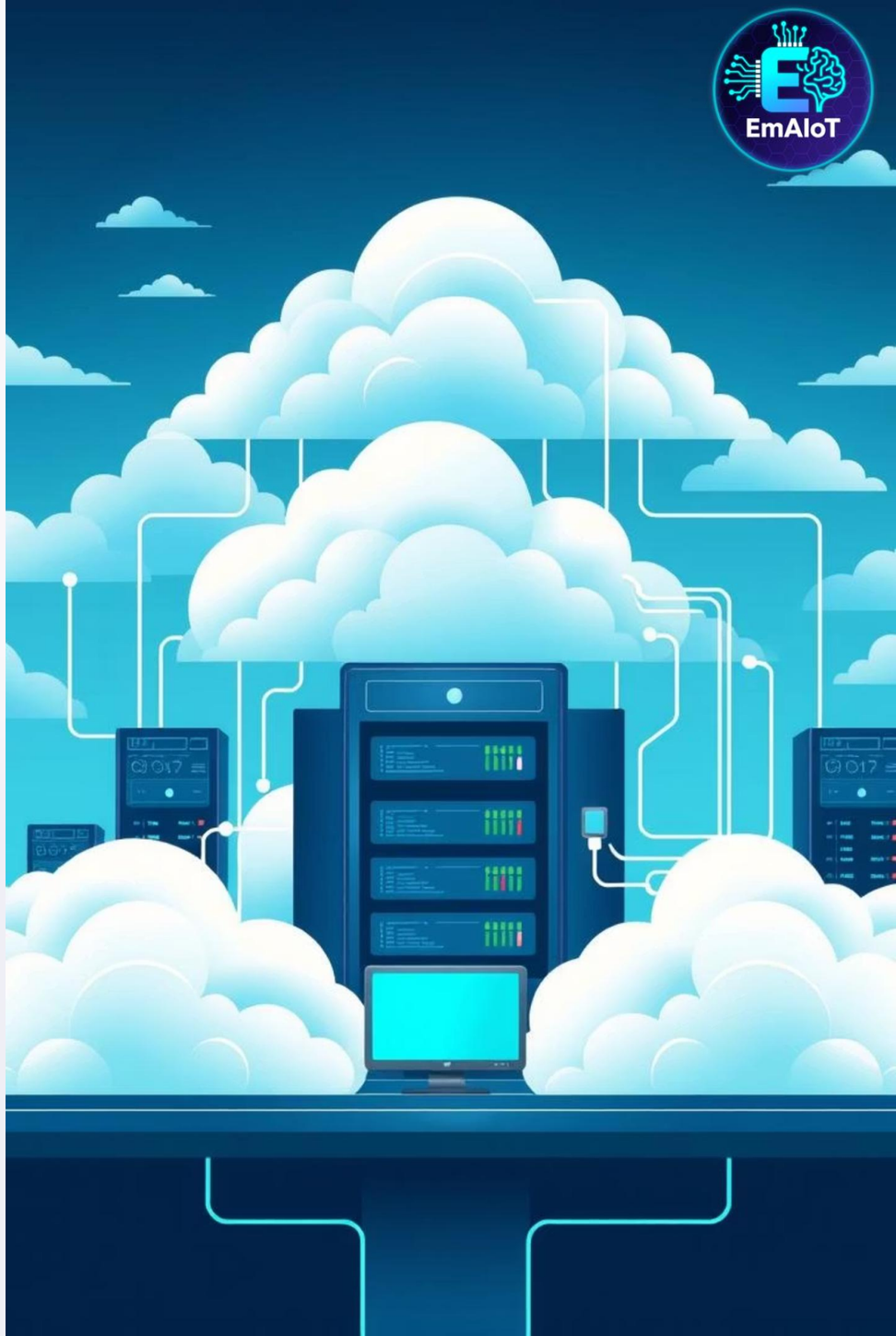
- Device registration and management
- Data ingestion from thousands of devices
- Data storage in databases (InfluxDB, MongoDB)
- Rules engine execution ("if temp > 30°C, send alert")
- Analytics and machine learning
- API endpoints for applications

Popular platforms:

- AWS IoT Core: Enterprise-grade, scalable
- Microsoft Azure IoT Hub: Device twins, DPS
- Google Cloud IoT: BigQuery integration
- Firebase: Real-time database, easy setup
- ThingsBoard: Open-source, self-hosted

Real example: AWS IoT setup

- Device connects via MQTT
- Data stored in DynamoDB
- Lambda function processes rules
- SNS sends SMS alerts



6. Cloud Applications: The User Interface 📱💻

The customer interface for dashboards, settings, and device control.

Types of applications:

Mobile apps (iOS/Android)

Control devices on-the-go

Web dashboards (Grafana, custom)

Monitor data and trends

Voice assistants (Alexa, Google)

Voice control integration

Email/SMS notifications

Automated alerts

Third-party integrations

IFTTT, Zapier

Main features:

- Real-time data visualization (charts, graphs)
- Device control (on/off switches, sliders)
- Historical data analysis
- User management and permissions
- Alert configuration

Real examples:

- Smart home app: Control lights, view energy usage
- Industrial dashboard: Monitor machine performance
- Agricultural app: Track soil moisture, weather